Atividade 5 – Sistema Distribuído de Classificação Inteligente de Mensagens com RabbitMQ

Desenvolver um sistema distribuído utilizando **Java, Docker, RabbitMQ e Inteligência Artificial**, capaz de gerar, distribuir e classificar mensagens automaticamente por meio de dois consumidores independentes.

Como deve funcionar (4 containers)



1. Gerador de Mensagens

- Gerar continuamente pelo menos **5 mensagens por segundo**.
- Produzir mensagens textuais de dois tipos:
  - **Avaliações de produtos:** comentários positivos, negativos ou neutros.
  - **Solicitações de suporte:** relatos de problemas técnicos que devem ser classificados por categoria.
- Publicar as mensagens no RabbitMQ, identificando seu tipo.

2. RabbitMQ

- Utilizar um **Topic Exchange** para encaminhar mensagens aos consumidores.
- Utilizar as routing keys `review.product` e `support.request`.
- Criar uma fila específica para cada consumidor.
- Habilitar a interface de administração para acompanhar a quantidade de mensagens nas filas.

3. Consumidor 1 – Análise de Sentimento

- Consumir mensagens de avaliações de produtos.
- Utilizar uma IA embarcada para classificar cada avaliação como **positiva, negativa ou neutra**.
- Exibir o resultado da classificação no console.
- Utilizar preferencialmente a biblioteca Smile para Java.

4. Consumidor 2 – Classificação de Solicitações

- Consumir mensagens de solicitações de suporte.
- Utilizar uma IA embarcada para classificar cada solicitação em uma das categorias:
  - Problema de acesso.
  - Problema de desempenho.
  - Problema de conexão.

- Exibir o resultado da classificação no console.
- Utilizar preferencialmente a biblioteca Smile para Java.

Regras técnicas

- Todos os serviços devem ser executados em containers Docker, conectados à mesma rede.
- O RabbitMQ deverá utilizar Topic Exchange.
- Cada consumidor deverá possuir seu próprio mecanismo de classificação baseado em IA.
- Os consumidores deverão processar as mensagens mais lentamente do que a taxa de chegada, permitindo visualizar o crescimento das filas.
- As mensagens deverão conter um **identificador único, data/hora de geração, tipo e conteúdo**.
- O sistema deverá implementar confirmação manual de consumo (*manual ACK*).
- Mensagens que apresentarem erro de processamento deverão ser encaminhadas para uma **Dead Letter Queue (DLQ)**, em vez de serem descartadas.
- A configuração dos serviços deverá ser realizada por meio do Docker Compose.

Demonstração

Durante a execução, os alunos deverão demonstrar:

1. A geração contínua de mensagens.
2. O encaminhamento correto das mensagens pelo Topic Exchange.
3. A classificação realizada pelas duas IAs.
4. O crescimento das filas na interface do RabbitMQ.
5. O tratamento de uma mensagem com erro por meio da DLQ.

Entrega

Postar no SIGAA apenas o **link do repositório GitHub**, contendo:

- Código-fonte dos serviços.
- Dockerfile dos serviços Java e arquivo Docker Compose.
- README com arquitetura, instruções de execução e descrição das IAs.
- Evidências da execução, incluindo capturas da interface do RabbitMQ.
- Um vídeo de **+-5 minutos** demonstrando o funcionamento do sistema.